

Case-Based Calibration of Adaptive Reasoning and Execution for LLM Tool Use

Renning Pang^{}, Tian Lan^{}, Leyuan Liu()^{}, Piao Tong^{}, Sheng Cao^{}, and
Xiaosong Zhang^{}

University of Electronic Science and Technology of China, Chengdu 611731, China
{prn,piaot}@std.uestc.edu.cn,
{lantian1029,leyuanliu,caosheng,johnsonzxs}@uestc.edu.cn

Abstract. Tool use extends large language models beyond parametric knowledge, but reliable execution requires balancing appropriate reasoning depth with strict structural validity. We approach this problem from a case-based perspective to present CAST, a case-driven framework that treats historical execution trajectories as structured cases. Instead of reusing raw exemplar outputs, CAST extracts case-derived signals to identify complexity profiles for estimating optimal reasoning strategies, alongside failure profiles to map likely structural breakdowns. The framework translates this knowledge into a fine-grained reward design and adaptive reasoning, enabling the model to autonomously internalize case-based strategies during reinforcement learning. Experiments on BFCLv2 and ToolBench demonstrate that CAST improves both schema-faithful execution and task-level tool-use success while reducing unnecessary deliberation. The approach achieves up to 5.85 percentage points gain in overall execution accuracy and reduces average reasoning length by 26%, significantly mitigating high-impact structural errors. Ultimately, this demonstrates how historical execution cases can provide reusable adaptation knowledge for calibrated tool use.

Keywords: Case-Based Reasoning · Large Language Models · Tool Use
· Reinforcement Learning · Reasoning Budget Calibration

1 Introduction

Tool use has become a central mechanism for extending large language models (LLMs) beyond parametric knowledge, and recent surveys identify tool invocation, planning, and action coordination as core components of LLM-agent systems [25,4,6,21]. At the same time, benchmark-oriented studies suggest that reliable function calling remains far from solved, especially in settings that require memory, dynamic decision-making, and long-horizon reasoning rather than single-turn text generation alone [22]. Representative frameworks such as Toolformer [26], ReAct [29], and ToolLLM [24] further show that the key challenge is no longer simply whether LLMs can call tools, but whether they can allocate reasoning effort and execute structured tool actions in a stable and task-sensitive manner.

This challenge arises because tool use places two simultaneous demands on the model. Different queries require different amounts of intermediate reasoning:

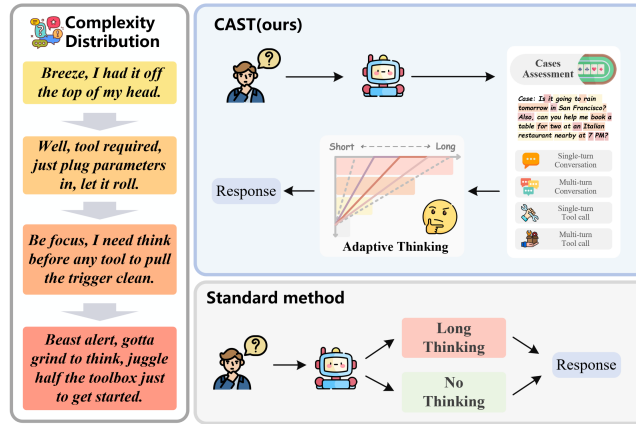


Fig. 1. Comparison of reasoning workflows with and without a case-based mechanism.

some can be solved with little deliberation, whereas others require additional reasoning to verify constraints, normalize arguments, or compose multi-step tool calls. Recent work on adaptive chain-of-thought (CoT) and efficient tool calling suggests that treating all inputs with the same reasoning policy is often inefficient and can even be counterproductive [20,19,27]. At the same time, tool execution is governed by strict structural constraints, while standard reinforcement learning (RL) often provides only coarse end-task feedback, making it difficult to identify whether failure originates from tool selection, parameter coverage, type mismatch, schema violation, or value construction. The central difficulty in tool use is therefore to calibrate reasoning depth and execution structure jointly under heterogeneous task demands.

We approach this problem from a modern Case-Based Reasoning (CBR) perspective on tool-use adaptation in LLMs. In our setting, historical execution trajectories are treated as cases that record how a task was solved, how much reasoning preceded action, whether the tool invocation succeeded, and what kinds of failures arose when it did not. This view is consistent with classical CBR [1] and with subsequent work emphasizing adaptation bottlenecks and case-base maintenance [11,7]. Recent studies further suggest that LLMs can support several stages of the CBR process, including case adaptation, similarity assessment, and experience-grounded agent reasoning [13,14,18,3]. Together, these developments motivate the use of historical execution cases as structured sources of calibration knowledge for tool use. In this paper, we formulate tool use as a case-based adaptation problem and introduce **Case-driven Adaptation for Schema-faithful Tool use (CAST)**, a case-driven framework that extracts two forms of case-derived signals from past trajectories. Specifically, it derives a complexity profile to estimate the necessary reasoning depth and a failure profile to map likely structural breakdowns. As shown in Figure1, rather than imposing a static reasoning limit or a uniform chain-of-thought policy, CAST translates this case-derived knowledge into a fine-grained reward design that supports adaptive reasoning. In particular, the model learns to shorten deliberation for easy cases while pre-

serving sufficient reasoning steps for cases that require constraint verification, argument normalization, or multi-step tool composition. This approach enables the model to internalize historical case experiences during reinforcement learning, empowering it to autonomously orchestrate the model’s reasoning budget and execute schema-faithful tool actions. Experiments on BFCLv2 and ToolBench show that CAST improves both schema-faithful execution and task-level tool-use success while reducing unnecessary reasoning, with especially strong gains on more complex cases and clear reductions in high-impact structural failures.

The remainder of this paper is organized as follows. Section 2 reviews related work on CBR for LLMs and agents, adaptive reasoning, and tool-use alignment. Section 3 formulates tool use as a case-based adaptation problem. Section 4 presents the CAST framework. Section 5 reports the experimental results and analyses. Section 6 concludes the paper and discusses limitations and future work.

2 Related Work

Case-based reasoning (CBR) addresses new problems by reusing and adapting solutions from similar prior cases through retrieval, reuse, revision, and retention [1]. In recent years, this perspective has increasingly intersected with large language models and agentic systems. Prior work has explored the use of LLMs for case adaptation in argumentative reasoning [13], while more recent studies investigate how LLMs can support similarity assessment in case retrieval [14]. Other work extends case-based ideas to reinforcement learning and agent settings, for example through case-based knowledge distillation for reinforcement learning [18], and unified experience-grounded agentic reasoning architectures [3]. Recent work has also examined how LLMs can support case-base population from unstructured sources [10] and how CBR can be integrated with LLMs in practical decision-support settings such as fraud detection [9]. Taken together, these studies suggest that LLMs can support multiple stages of the CBR process, including similarity assessment, adaptation, case acquisition, and the use of structured prior experience in agent reasoning.

A related line of research studies how to regulate the amount of reasoning generated by large language models. Existing methods seek to shorten, compress, or selectively trigger chain-of-thought in order to improve efficiency while maintaining competitive task performance [20,19]. This line of work is directly relevant to our setting because it recognizes that different inputs may require different amounts of intermediate reasoning. In parallel, another line of work focuses on tool use and reasoning-acting behavior in LLMs. Toolformer shows that language models can learn when and how to invoke external tools [26], while ReAct demonstrates the value of interleaving reasoning traces with external actions [29]. Tool-use research has also been accompanied by increasingly realistic evaluation settings, including large-scale API benchmarks and function-calling leaderboards [24,22]. More recent work further improves invocation discipline and efficiency, for example through meta-cognitive triggering [15], alignment for efficient tool calling [27], or token-level policy gradient reshaping for tool-use

LLMs [16]. These studies substantially advance reasoning control and tool-use alignment, but they generally treat reasoning depth and execution structure as separate concerns rather than as jointly case-conditioned aspects of the same problem.

Our work is motivated by the gap between these directions. Existing CBR-oriented research suggests that historical cases can provide reusable structure for future reasoning and action, while adaptive reasoning and tool-alignment research shows that both reasoning cost and execution discipline must be carefully controlled. What remains underexplored is how historical execution cases can be used to derive calibration signals for these two aspects jointly. CAST is a case-driven framework in which historical execution cases provide case-derived complexity and failure signals, designed to explicitly internalize the case-based experience that calibrates reinforcement learning through reasoning-budget control and schema-faithful tool optimization.

3 Problem Formulation

Given a user query q and a tool set $\mathcal{T}$, tool use requires a model to generate a trajectory $\tau = (r, c)$, where r denotes the reasoning trace and $c = \text{call}(f, z)$ denotes a structured tool invocation with function f and arguments z . We formulate this problem from a case-based adaptation perspective: historical execution trajectories are treated as execution cases, $\xi_i = (q_i, r_i, c_i, o_i, \phi_i)$, where q_i is the input query, r_i is the reasoning trace, c_i is the executed tool call, o_i is the execution outcome, and $\phi_i = (h_i, e_i)$ is the case profile consisting of a complexity profile h_i and a failure profile e_i . Under this formulation, historical executions are not treated merely as supervision traces, but as structured cases from which case-derived calibration signals can be obtained for reasoning-budget control and schema-faithful tool use. The objective is therefore to learn a policy π_θ calibrated by these historical cases, such that the generated trajectory $\tau \sim \pi_\theta(\cdot \mid q, \mathcal{T})$ produces tool actions that are both semantically appropriate and structurally executable.

4 Method

Figure 2 illustrates the overall architecture of CAST. From a case-based perspective, CAST organizes historical tool-use trajectories as structured execution cases, derives case-derived complexity and failure signals from these cases, and uses them to guide two coordinated adaptation processes: reasoning-budget calibration and schema-faithful tool optimization. In implementation, CAST is trained with a supervised warm-up stage followed by reinforcement learning calibrated by these case-derived signals. In this sense, reasoning-budget calibration model is encouraged to vary its deliberation depth according to case complexity before producing a schema-faithful tool action. Under this formulation, reinforcement learning serves as the optimization mechanism, while the methodological core lies in case-driven capability assessment and adaptation.

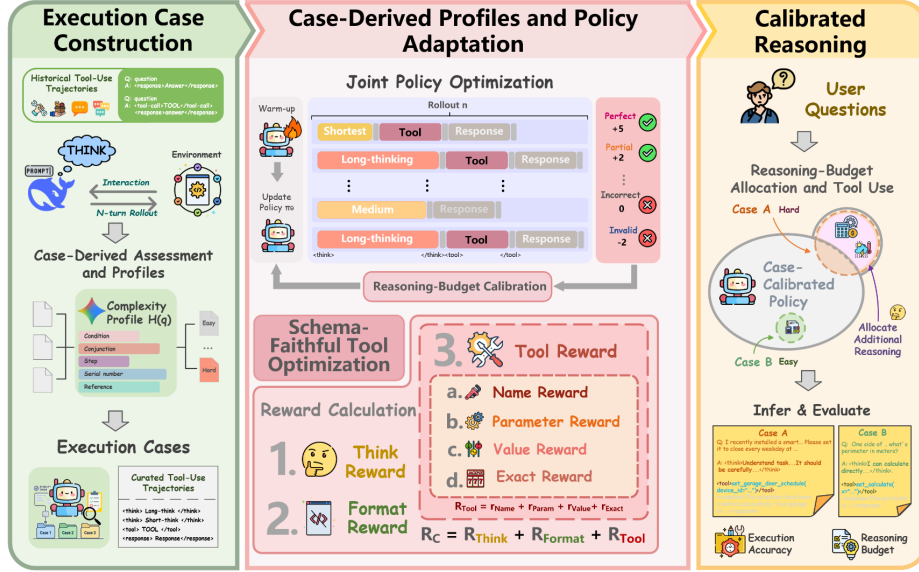


Fig. 2. Overview of CAST. Historical tool-use trajectories are organized as execution cases. These case signals guide two coordinated adaptation processes: reasoning-budget calibration and schema-faithful tool optimization. Reinforcement learning serves as the optimization mechanism for this case-based adaptation framework.

4.1 Execution Case Construction

CAST first converts historical tool-use trajectories into execution cases. Unlike ordinary supervision pairs, an execution case preserves not only the input query and target tool call, but also the intermediate reasoning process and the observed execution outcome. This allows historical trajectories to function as structured experience rather than isolated input–output examples. Formally, each case is represented as $\xi_i = (q_i, r_i, c_i, o_i, \phi_i)$, where q_i is the user query, r_i is the reasoning trace, c_i is the structured tool invocation, o_i is the execution outcome, and ϕ_i is the case profile defined in Section 4.2. In practice, we bootstrap the case base from tool-augmented reasoning trajectories distilled from DeepSeek-R1, normalize all tool interactions into a unified function-schema format, and filter malformed or unverifiable traces during preprocessing.

4.2 Case Representation: Complexity and Failure Profiles

After constructing the execution case base, CAST represents each case through a profile $\phi_i = (h_i, e_i)$, where h_i is a complexity profile and e_i is a failure profile. The complexity profile characterizes how much intermediate reasoning is typically required before action, whereas the failure profile characterizes where execution is most likely to break, such as function selection, argument construction, type mismatch, or schema violation. Together, these profiles provide the case-derived calibration signals used in the subsequent adaptation stages.

We estimate case complexity from the base model’s observed execution behavior rather than from a manually specified heuristic. For each historical query–target

pair (q, a) , we sample a trajectory $y \sim \pi_{\text{base}}(\cdot | q)$ and evaluate it with the verifier V . If the generated tool call is fully correct and schema-faithful, the case is assigned zero hardness. Otherwise, the failed trajectory is assessed by an external rubric-guided judge using reference examples of simple and difficult cases. In our implementation, this judge is instantiated with Gemini-Pro. We define the resulting hardness score as

$$H(q) = \begin{cases} 0, & V(y, a) = 1, \\ 1 - S_{\text{judge}}(q, y), & V(y, a) = 0, \end{cases} \quad (1)$$

where $S_{\text{judge}}(q, y) \in [0, 1]$ denotes the judged adequacy of the failed trajectory with respect to logical coherence, parameter construction, and schema adherence. We use this score as an operational proxy for reasoning demand, and partition the case base into D_{easy} and D_{hard} for curriculum scheduling while retaining the continuous score for finer-grained analysis.

The failure profile is derived from the mismatch between the generated tool call and the reference execution. For each failed case, we record the principal failure dimensions involved in the mismatch, including function-name errors, parameter-key omissions, type mismatches, constraint violations, and value mismatches, and summarize them as

$$e_i = (e_i^{\text{name}}, e_i^{\text{key}}, e_i^{\text{type}}, e_i^{\text{constraint}}, e_i^{\text{value}}). \quad (2)$$

Each component indicates whether the corresponding failure pattern is present in the trajectory.

4.3 Case-Based Adaptation for Reasoning-Budget Calibration

The adaptation problem in tool use is to determine how much reasoning should precede action for a given case. A uniform reasoning policy is inadequate because different cases require different amounts of intermediate deliberation: simple cases may suffer from unnecessary verbosity. CAST addresses this problem by using the case complexity profile to internalize historical execution strategies into the model’s policy. Through a fine-grained reward design, the model learns to autonomously produce appropriately short or long CoT traces, mirroring the successful adaptation patterns found in similar prior cases.

Concretely, the complexity profile introduced in Section 4.2 is operationalized by the hardness score $H(q) \in [0, 1]$, where smaller values indicate easier cases and larger values indicate more difficult ones. We discretize $H(q)$ into a small number of difficulty bands $d(q)$ only for estimating stable empirical length baselines, while retaining the continuous value $H(q)$ itself as the control signal for reward shaping. Let $r_a \in [-2, 2]$ denote the final answer score and L the response length. We define a difficulty-conditioned length baseline as:

$$L_{\text{emp}}^{d(q)}(t) = L_{\text{max}}^{d(q)} - (L_{\text{max}}^{d(q)} - L_{\text{target}}^{d(q)}) \cdot \min\left(1, \frac{t}{T_{\text{warmup}}}\right), \quad (3)$$

where $L_{\text{max}}^{d(q)}$ is the initial relaxed length limit for difficulty band $d(q)$, $L_{\text{target}}^{d(q)}$ is the target concise length, and T_{warmup} is the curriculum duration.

We then define the excess-length ratio

$$\rho(q, L) = \max\left(0, \frac{L}{L_{emp}^{d(q)}} - 1\right), \quad (4)$$

which measures how much the current response exceeds the expected reasoning budget, and the complexity-sensitive gating weight

$$\lambda(q) = 1 - H(q), \quad (5)$$

so that easy cases are more strongly penalized for unnecessary overthinking, whereas difficult cases are less sensitive to length and remain primarily correctness-driven.

The shaping coefficient is defined as

$$\alpha(q, r_a, L) = \begin{cases} \max(0, 1 - \lambda(q)\rho(q, L)), & r_a > 0, \\ 1 + \lambda(q)\rho(q, L), & r_a < 0, \\ 1, & r_a = 0, \end{cases} \quad (6)$$

and the reasoning-side reward is

$$\mathcal{R}_{\text{Think}} = \alpha(q, r_a, L) \cdot r_a. \quad (7)$$

Under this formulation, the same complexity signal plays two roles. Through $d(q)$, it provides a stable empirical baseline for expected reasoning length; through $H(q)$, it determines how strongly overlong reasoning should be penalized. As a result, easy cases are encouraged to remain concise, while difficult cases preserve a larger reasoning workspace and are optimized primarily with respect to answer correctness.

4.4 Failure-Profile-Grounded Optimization for Schema-Faithful Tool Use

While reasoning-budget calibration governs how much deliberation precedes action, a complementary challenge remains: ensuring that the final tool action is structurally executable. Even when a reasoning trace appears semantically plausible, execution can still fail because of errors in function names, argument keys, types, constraints, or values. CAST therefore treats tool optimization as a separate but coordinated objective grounded in the failure profile $e_i = (e_i^{\text{name}}, e_i^{\text{key}}, e_i^{\text{type}}, e_i^{\text{constraint}}, e_i^{\text{value}})$. Let $\mathcal{G}$ and $\mathcal{P}$ denote the ground-truth and predicted collections of tool calls. To handle multi-call settings, we align them by maximum-weight bipartite matching:

$$\mathcal{J} = \arg \max_{M \in \mathcal{M}(\mathcal{G}, \mathcal{P})} \sum_{(G, P) \in M} s_{\text{match}}(G, P), \quad (8)$$

where $s_{\text{match}}(G, P) = \delta(\text{name}(G), \text{name}(P)) + \frac{|K^G \cap K^P|}{|K^G \cup K^P|}$, and K^G, K^P are the parameter-key sets of G, P . Repeated tool calls remain distinct nodes in the matching graph.

We then define a six-dimensional structural reward vector:

$$\mathbf{r}_{\text{tool}} = (r_{\text{name}}, r_{\text{key}}, r_{\text{type}}, r_{\text{constraint}}, r_{\text{value}}, r_{\text{exact}})^\top. \quad (9)$$

Let N_G, N_P be the overall function-name sets of $\mathcal{G}, \mathcal{P}$. The name score is the Jaccard overlap $r_{\text{name}} = \frac{|N_G \cap N_P|}{|N_G \cup N_P|}$. The key score averages parameter-key overlap over aligned calls: $r_{\text{key}} = \frac{1}{|\mathcal{J}|} \sum_{j \in \mathcal{J}} \frac{|K_j^G \cap K_j^P|}{|K_j^G \cup K_j^P|}$ when $|\mathcal{J}| > 0$, and 0 otherwise; when $K_j^G = K_j^P = \emptyset$, the corresponding overlap term is set to 1. We define $r_{\text{type}}, r_{\text{constraint}}, r_{\text{value}}$ as the average indicator matches over overlapping keys, with default value 0 when the number of overlapping keys is zero. The exact-match term is $r_{\text{exact}} = V_{\text{AST}}(\mathcal{P}, \mathcal{G}) \in \{0, 1\}$, where AST denotes Abstract Syntax Tree matching. The raw structural score is:

$$R_{\text{raw}} = r_{\text{name}} + r_{\text{key}} + r_{\text{type}} + r_{\text{constraint}} + r_{\text{value}} + r_{\text{exact}}. \quad (10)$$

Since each component is bounded in $[0, 1]$, the global maximum is $S_{\text{max}} = 6$. We therefore define the final tool-side reward as

$$\mathcal{R}_{\text{Tool}} = 2 \cdot \frac{R_{\text{raw}}}{S_{\text{max}}} - 1 \in [-1, 1]. \quad (11)$$

As a result, CAST provides dense and interpretable credit assignment for schema-faithful tool use without hand-tuned gating rules or query-dependent scaling constants.

4.5 Training Objective and Optimization

Given a query q and a tool set $\mathcal{T}$, the model generates a trajectory $\tau = (r, c)$. To guide the optimization securely and prevent degenerate behaviors, we formulate a composite reward that provides feedback at three complementary levels of abstraction:

$$\mathcal{R}_C = \mathcal{R}_{\text{Think}} + \mathcal{R}_{\text{Format}} + \mathcal{R}_{\text{Tool}}. \quad (12)$$

The first term, $\mathcal{R}_{\text{Think}}$, is derived from the complexity profile and aligns the model toward adaptive reasoning by calibrating the intermediate deliberation budget for each case. (Section 4.3). The second, $\mathcal{R}_{\text{Format}}$, is a rule-based guardrail that enforces valid tag encapsulation. The third, $\mathcal{R}_{\text{Tool}}$, relies on the failure profile to evaluate fine-grained schema-faithful accuracy (Section 4.4).

In practice, we first initialize the model using SFT on curated tool-augmented trajectories with the standard next-token prediction loss:

$$\mathcal{L}_{\text{SFT}} = -\frac{1}{|\mathcal{D}_{\text{SFT}}|} \sum_{(x, y) \in \mathcal{D}_{\text{SFT}}} \sum_{t=1}^{|y|} \log \pi_\theta(y_t \mid x, y_{<t}). \quad (13)$$

After initialization, we optimize the policy π_θ using a Group Relative Policy Optimization (GRPO)-based procedure augmented with the case-derived composite reward $\mathcal{R}_C$, which replaces the standard scalar reward with our decomposed complexity- and failure-conditioned signals. To stabilize the learning dynamics,

we organize the RL training following an Easy-to-Hard curriculum based on the difficulty subsets partitioned by the complexity profile. For each query q , we sample a group of G trajectories $\{\tau_1, \dots, \tau_G\}$. The group-relative advantage for the i -th trajectory is computed by normalizing the composite rewards within the group:

$$A_i = \frac{\mathcal{R}_C(\tau_i) - \mu_{\mathcal{R}}}{\sigma_{\mathcal{R}} + \epsilon_{\text{stab}}}, \quad (14)$$

where $\mu_{\mathcal{R}}$ and $\sigma_{\mathcal{R}}$ are the mean and standard deviation of the group rewards, and ϵ_{stab} is a small constant for numerical stability. The policy is then updated by maximizing the clipped surrogate objective augmented with a KL divergence penalty:

$$\mathcal{J}_{\text{CAST}}(\theta) = \mathbb{E}_{q, \{\tau_i\}_{i=1}^G} \left[\min \left(\rho_i A_i, \text{clip}(\rho_i, 1 - \epsilon, 1 + \epsilon) A_i \right) - \beta \cdot D_{\text{KL}} \left(\pi_{\theta}(\cdot | q) \parallel \pi_{\text{ref}}(\cdot | q) \right) \right], \quad (15)$$

where $\rho_i = \frac{\pi_{\theta}(\tau_i | q)}{\pi_{\theta_{\text{old}}}(\tau_i | q)}$ is the probability ratio, ϵ is the clipping margin, β controls the KL penalty strength, and π_{ref} is the reference policy. Under this formulation, SFT provides warm-up initialization and the GRPO-based procedure serves as the optimization engine, while the methodological core of CAST lies in representing historical executions as structured cases, deriving complexity and failure profiles, and translating them into the decomposed case-derived signals that drive the GRPO updates.

5 Experiments

5.1 Experimental Setup

We evaluate CAST on BFCLv2, including 5,551 instances covering single-turn, parallel, multi-step, and irrelevance-detection scenarios, and on ToolBench with diverse queries mapping to REST APIs across 49 categories. Model-level baselines include the original backbones alongside their SFT and GRPO variants. We primarily use Qwen2.5-7B-Instruct, adding Qwen2.5-Coder-7B-Instruct and Llama-3.2-8B-Instruct to examine cross-backbone robustness, and report closed models (GPT-4o [12], Qwen-Max [28], DeepSeek-V3 [17]) for context. Method-level comparisons feature Toolformer [26], ReAct [29], ToolAlign [5], CoT-Valve [20], OTC [8], Granite [2], and Gorilla [23] evaluated under identical protocols. We utilize CAST via Megatron with an 8K maximum response length and evaluate inference using SGLang. To balance exploration and warm-up, we train for 2 epochs during the SFT phase. For RL training, we sample a group of $G = 8$ rollouts per query at a temperature of 0.9. The policy is optimized using AdamW with a peak learning rate of 1×10^{-6} , a cosine learning rate scheduler, and a weight decay of 0.01. To stabilize the reinforcement learning dynamics, we apply a KL divergence penalty coefficient of $\beta = 0.01$.

Table 1. Main results on BFCLv2 and ToolBench. We report execution accuracy on BFCLv2 (Non-Live AST, Live, and Overall) and task success on ToolBench (Pass and Win).

Category	Model / Method	Non-Live	AST	Live	Overall	Pass	Win
		BFCLv2 (%)			ToolBench (%)		
<i>Closed Source</i>	GPT-4o	86.83	78.92	82.88	64.43	67.63	
	Qwen-Max	84.97	80.85	82.91	70.93	71.83	
	DeepSeek-V3	86.37	75.28	80.83	70.73	72.93	
<i>Open Source</i>	Llama-3.2-8B-Instruct-SFT	83.14	73.82	78.48	70.63	45.83	
	Llama-3.2-8B-Instruct-GRPO	82.18	75.21	78.73	71.23	45.23	
	Llama-3.2-8B-Instruct-CAST	83.95	76.84	80.43	75.93	47.23	
	Qwen2.5-Coder-7B-Instruct-SFT	86.07	74.92	80.86	67.84	65.37	
	Qwen2.5-Coder-7B-Instruct-GRPO	86.51	75.07	80.79	71.13	67.82	
	Qwen2.5-Coder-7B-Instruct-CAST	87.12	82.43	84.70	80.61	79.14	
	Qwen2.5-7B-Instruct-SFT	86.23	78.90	82.58	68.67	65.23	
	Qwen2.5-7B-Instruct-GRPO	87.05	80.29	83.67	72.71	68.23	
	Qwen2.5-7B-Instruct-CAST	88.24	87.40	88.43	80.67	79.43	
	Granite	86.17	79.19	84.71	68.47	50.17	
	Gorilla	86.02	80.44	82.21	62.27	46.29	
	Toolformer	76.11	59.47	67.07	48.92	22.11	
<i>Other</i>	ReAct	73.58	58.43	66.08	43.37	18.22	
	ToolAlign	77.26	61.47	71.16	46.78	22.36	
	OTC	82.64	71.28	78.33	65.49	36.12	

5.2 Overall Performance and Efficiency

Table 1 reports the main results on BFCLv2 and ToolBench. On BFCLv2, CAST consistently improves over both SFT and GRPO across all three open-source backbones, indicating that case-derived calibration transfers across different model families and pretraining styles. The strongest result is obtained with Qwen2.5-7B-Instruct, where CAST reaches 88.43% overall, gaining 5.85 points over SFT and 4.76 points over GRPO. The narrow 0.84-point gap between Non-Live AST and Live scores suggests that the gains transfer from offline schema matching to actual tool execution. The same trend is visible on the other open-source backbones. On Llama-3.2-8B-Instruct, CAST improves BFCLv2 Overall from 78.48% under SFT and 78.73% under GRPO to 80.43%. On Qwen2.5-Coder-7B-Instruct, CAST reaches 84.70% overall, with especially large gains on Live execution, rising from 74.92% and 75.07% to 82.43%. Together, these results suggest that CAST is particularly effective when evaluation places stricter weight on execution validity. On ToolBench, the transfer pattern is also consistently positive. CAST achieves the strongest ToolBench results among CAST-trained models with Qwen2.5-7B-Instruct, reaching 80.67% Pass and 79.43% Win, both substantially above its SFT and GRPO counterparts. Llama-3.2-8B-Instruct shows a similar trend. Notably, the updated results on Qwen2.5-Coder-7B-Instruct now reveal a strong monotonic improvement as well: ToolBench Pass increases from 67.84% under SFT and 71.13% under GRPO to 80.61% under CAST, while Win rises from 65.37% and 67.82% to 79.14%. Rather than indicating a trade-off between schema-faithful calibration and end-to-end task success, the coder backbone now provides additional evidence that case-derived calibration transfers beyond local structural correctness and yields substantial gains on task-level tool-use performance.

Table 2. Ablation of reasoning-side and tool-side adaptation on BFCLv2.

Variant	Non-Live AST $\uparrow$ Avg. Reasoning Length (tok) $\downarrow$
base-GRPO	87.05 236.9
dynamic-only	85.50 164.7
schema-only	86.59 214.5
CAST	88.24 175.4

5.3 Evidence for Case-Conditioned Adaptation

We evaluate CAST through ablations, budget sensitivity, and training stability. Figure 3 shows that removing any major component consistently lowers average reward across backbones, indicating that CAST depends on the interaction between reasoning-side and tool-side adaptation. Table 2 confirms this on BFCLv2. The dynamic-only variant, which keeps the adaptive reasoning-budget component without the full schema-level reward, reduces average length from 236.9 to 164.7 tokens while reaching 85.50 Non-Live AST. This confirms that the adaptive reasoning component alone can suppress unnecessary deliberation, although schema-faithful optimization is still needed for the best execution accuracy. Combining both gives the best result 88.24% Non-Live AST at 175.4 tokens. Table 3 shows the importance of budget control. Removing the cap increases average length to 486.2 tokens and lowers BFCLv2 Overall to 86.3%. A strict 50th-percentile cap shortens outputs to 140.8 tokens but drops ToolBench Pass to 76.5%. The 80th-percentile setting achieves the best overall trade-off across BFCLv2 and ToolBench, supporting case-conditioned reasoning budgets for heterogeneous tool-use tasks.

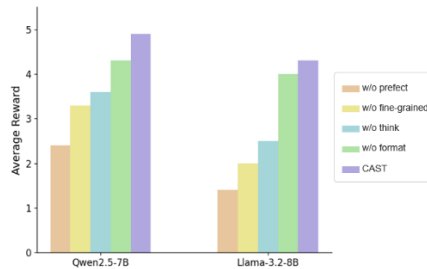
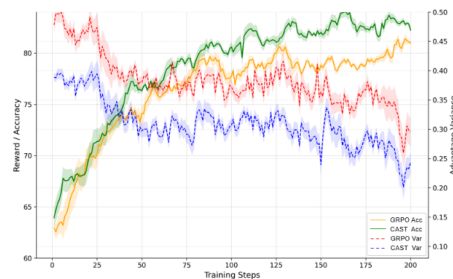
**Fig. 3.** Ablation study of the major adaptation components.**Fig. 4.** Training accuracy and normalized advantage variance.

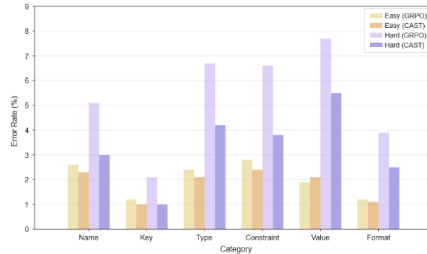
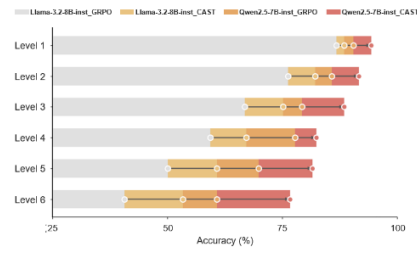
Figure 4 shows that CAST also stabilizes RL training. Its normalized advantage variance decreases from 0.48 to 0.10, while GRPO remains around 0.21, indicating cleaner credit assignment and fewer oscillations between overthinking and underthinking.

5.4 Case Complexity and Failure Profiles

To verify that the case-derived profiles operate as intended, we analyze their impact across three dimensions: instance-level budget allocation, structural error suppression, and global curriculum organization.

Table 3. Impact of varying reasoning length budgets on task accuracy and schema alignment.

Threshold	BFCLv2 (Overall)	ToolBench (Pass)	ToolBench (Win)	Length
0 (no cap)	86.3	69.9	65.7	486.2
100th (loose)	87.5	80.3	79.6	240.9
80th (default)	88.43	80.67	79.43	175.4
50th (strict)	86.8	76.5	73.1	140.8

**Fig. 5.** Performance across fine-grained difficulty levels.**Fig. 6.** Error-rate distribution by category.

Adaptive Budget Allocation via Complexity Profiles. Figure 5 reports execution accuracy across fine-grained difficulty levels. If the complexity profile merely acted as a length penalty, its effect would be roughly uniform across buckets or even harmful on harder cases. Instead, CAST yields only small gains on easy instances, where budget control mainly reduces verbosity, but substantially larger gains as difficulty increases. This pattern suggests that the complexity profile enables adaptive reasoning allocation: CAST shortens reasoning when additional deliberation is unnecessary, but preserves or expands the reasoning budget when compositional reasoning is required.

Structural Error Suppression via Failure Profiles. We next study whether schema-faithful tool optimization targets specific failure modes. Figure 6 separates execution errors into structural violations and localized content errors. As shown in the Reward Calculation module of Figure 2, $\mathcal{R}_{\text{Tool}}$ explicitly penalizes structural deviations. Accordingly, CAST reduces structural failures much more than the GRPO baseline, leaving most residual errors in localized value prediction. This result indicates that CAST improves the conversion of free-form reasoning into schema-compliant tool execution.

Table 4. Effect of curriculum strategy on ToolBench.

Method	Pass (%)	Win (%)	Length
No Selection	73.2	69.5	2 417.3
Two Stage	76.8	74.2	297.3
Hard to Easy	68.5	64.3	426.3
Easy to Hard	80.7	79.4	175.4

At the training level, Table 4 tests whether the case-derived complexity score $H(q)$ provides a useful curriculum signal. The easy-to-hard schedule achieves the best accuracy and the shortest outputs, with an average generation length of 175.4 tokens. In contrast, the hard-to-easy schedule performs worst and produces much longer outputs (426.3 tokens on average), suggesting that early exposure to difficult cases encourages unstable trial-and-error behavior and persistent overthinking. Overall, case-derived complexity serves as an effective signal for both local budget control and global curriculum design.

Easy Instance Case			
Query: Could you check the current weather conditions in Beijing and New York City for me?			
SFT	The user asks for weather in Beijing and New York City. I should use <code>get_current_weather</code> for each location. Two calls, one for Beijing, one for NYC.	<code>get_current_weather</code> (location='Beijing') <code>get_current_weather</code> (location='NYC')	✓
GRPO	The query involves retrieving weather for two cities. I have access to <code>get_current_weather</code> . Since both Beijing and NYC are mentioned, I should invoke this for each city. No single API returns multiple cities. Wait, let me pause. Is parallel execution safe? While it reduces latency, there is a non-zero probability of hitting a 429 error if I trigger both at the same millisecond. Should I introduce a micro-delay? Or execute sequentially? Sequential adds latency but guarantees safety. However, the user likely values speed. No complex transformation needed, just passing city names. To maximize efficiency, I will call the function twice in parallel.	<code>get_current_weather</code> (location='Beijing') <code>get_current_weather</code> (location='NYC')	✓
CAST	The user wants weather for Beijing and NYC. The simplest approach is to call the weather function for each city. I will use <code>get_current_weather</code> twice, one for each city, possibly in parallel. No further reasoning needed, the task is straightforward.	<code>get_current_weather</code> (location='Beijing') <code>get_current_weather</code> (location='NYC')	✓

Fig. 7. Easy case.

Hard Instance Case			
Query: Calculate density (mass=50kg, volume=10m3), future value (\$5000 at 5% for 10 years), Apple stock price, and Amazon rating (ASIN: B08PPDJWC8)			
SFT	The request involves multiple tasks: density, future value, stock price, and product rating. I will tackle them in order. For density, I have <code>calculate_density</code> with mass=50, volume=10. For future value, <code>calculate_future_value</code> with present_value=5000, interest_rate=5, periods=10. Then Apple stock with <code>get_stock_price</code> , and Amazon rating with <code>get_rating_by_ASIN</code> .	<code>calculate_density</code> (mass=50, volume=10) <code>calculate_future_value</code> (5000, 5, 10) <code>get_stock_price</code> (‘Apple’) <code>get_rating_by_ASIN</code> (‘B08PPDJWC8’)	✗
GRPO	The user requests four independent results. The safest plan is to map each task to one tool call and validate arguments. For density: mass=50, volume=10, straightforward. For future value: present_value=5000, interest_rate=5%, periods=10. I need to decide how to encode 5%. Many functions expect decimal 0.05, but some accept 5. I’ll use 5, keeping closer to surface form. For Apple stock: the tool likely expects ticker symbol AAPL, not company name. For ASIN: pass B08PPDJWC8 exactly.	<code>calculate_density</code> (mass=50, volume=10) <code>calculate_future_value</code> (5000, 5, 10) <code>get_stock_price</code> (‘AAPL’) <code>get_rating_by_ASIN</code> (‘B08PPDJWC8’)	✗
CAST	The request comprises four diverse tasks requiring careful handling. I will approach each methodically. Density: <code>calculate_density</code> (mass=50, volume=10), straightforward. Future value: Must be precise with interest_rate. 5% = 0.05 decimal. Crucial to pass 0.05, not integer 5. Stock price: Safest to use ticker symbol AAPL, not company name. Amazon rating: Pass ASIN exactly as given.	<code>calculate_density</code> (mass=50, volume=10) <code>calculate_future_value</code> (5000, 0.05, 10) <code>get_stock_price</code> (‘AAPL’) <code>get_rating_by_ASIN</code> (‘B08PPDJWC8’)	✓

Fig. 8. Hard case.

5.5 Case Study

Figures 7 and 8 illustrate CAST’s adaptive reasoning-execution boundary. In the easy weather-query case, GRPO produces redundant deliberation about rate limits and execution order, whereas CAST directly emits the necessary tool calls. In the harder compositional case, SFT and GRPO copy the surface form of 5% into the tool arguments, while CAST preserves enough reasoning to normalize it to 0.05 and keeps the remaining calls structurally valid. This shows that CAST shortens reasoning when execution is straightforward and preserves it when semantic normalization is required.

6 Conclusion

This paper revisits tool use from a case-based reasoning perspective and proposes CAST, a framework that uses signals distilled from past execution cases to guide reinforcement learning. Specifically, CAST summarizes historical trajectories through complexity and failure profiles, and uses them to regulate reasoning length and supervise schema-level tool execution. Experiments on BFCLv2 and ToolBench show that this case-conditioned calibration improves execution accuracy, transfers to end-to-end tool-use success, and reduces unnecessary reasoning, with the largest gains on more complex cases. Long-horizon planning remains challenging, but the results suggest that case-derived supervision provides a practical basis for improving both reliable tool calling and downstream task completion. A natural next step is to extend this framework to richer case memories, stronger retrieval and reuse, and more interactive agent settings.

Acknowledgments. This work was partially supported by the National Natural Science Foundation of China (U2336204), Chengdu Industrial Chain Collaborative Innovation Project (Grant No. 2025-XT00-00017-GX) and the Open bidding for selecting the best candidates of Sichuan Provincial Department of Science and Technology (2024YFCY0003).

Generative AI Disclosure LLMs were used exclusively for stylistic refinement of the manuscript. All textual content was initially drafted in full by the authors and subsequently polished with the assistance of LLM-based tools, including ChatGPT and Gemini. All scientific contributions, technical methods, ideas, and core results presented in this work are entirely the original work of the authors.

References

1. Aamodt, A., Plaza, E.: Case-based reasoning: Foundational issues, methodological variations, and system approaches. *AI communications* **7**(1), 39–59 (1994)
2. Abdelaziz, I., Basu, K., Agarwal, M., Kumaravel, S., Stallone, M., Panda, R., et al.: Granite-function calling model: Introducing function calling abilities via multi-task learning of granular tasks. In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*. pp. 1131–1139 (2024)
3. Bergmann, R., Brand, F., Lenz, M., Malburg, L.: Exar: A unified experience-grounded agentic reasoning architecture. In: *International Conference on Case-Based Reasoning*. pp. 3–17. Springer (2025)
4. Chen, J., Wu, H., Pang, J., Wang, Y., Zhang, D., Sun, C.: Tool learning with language models: a comprehensive survey of methods, pipelines, and benchmarks. *Vicinearth* **2**(1), 16 (2025)
5. Chen, Z.Y., Shen, S., Shen, G., Zhi, G., Chen, X., Lin, Y.: Towards tool use alignment of large language models. In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. pp. 1382–1400 (2024)
6. Chow, S.S., Alvi, R., Rahman, S.S., Rahman, M.A., Raiaan, M.A.K., et al.: From language to action: a review of large language models as autonomous agents and tool users. *Artificial Intelligence Review* (2026)
7. Craw, S., Wiratunga, N., Rowe, R.C.: Learning adaptation knowledge to improve case-based reasoning. *Artificial intelligence* **170**(16-17), 1175–1192 (2006)
8. El-Tantawy, S., Abdulhai, B.: Towards multi-agent reinforcement learning for integrated network of optimal traffic controllers (marlin-otc). *Transportation Letters* **2**(2), 89–110 (2010)
9. Ge, X., Xu, J.: Integrating case-based reasoning with llm for expense fraud detection. In: *International Conference on Case-Based Reasoning*. pp. 52–66. Springer (2025)
10. Ghazouani, F., Giustozzi, F., Le Ber, F.: Llm-driven case-base populating for structuring and integrating restoration experiences. In: *International Conference on Case-Based Reasoning*. pp. 67–80. Springer (2025)
11. Hanney, K., Keane, M.T.: The adaptation knowledge bottleneck: How to ease it by learning from cases. In: *International Conference on Case-Based Reasoning*. pp. 359–370. Springer (1997)
12. Hurst, A., Lerer, A., Goucher, A.P., Perelman, A., Ramesh, A., Clark, A., Ostrow, A., et al.: Gpt-4o system card. *arXiv preprint arXiv:2410.21276* (2024)
13. Lenz, M., Bergmann, R.: Case-based adaptation of argument graphs with wordnet and large language models. In: *International Conference on Case-Based Reasoning*. pp. 263–278. Springer (2023)

14. Lenz, M., Hoffmann, M., Bergmann, R.: Llsim: large language models for similarity assessment in case-based reasoning. In: International Conference on Case-Based Reasoning. pp. 126–141. Springer (2025)
15. Li, W., Li, D., Dong, K., Zhang, C., Zhang, H., Liu, W., Wang, Y., Tang, R., Liu, Y.: Adaptive tool use in large language models with meta-cognition trigger. In: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). pp. 13346–13370 (2025)
16. Lin, Z., Wang, X., Cao, J., Chai, J.: Rest: Reshaping token-level policy gradients for tool-use large language models. In: ICLR 2026 Conference Proceedings (2026)
17. Liu, A., Feng, B., Xue, B., Wang, B., Wu, B., Lu, C., Zhao, C., Deng, C., Zhang, C., Ruan, C., et al.: Deepseek-v3 technical report. arXiv preprint arXiv:2412.19437 (2024)
18. Liu, H., Liu, Q., Wu, L., Shi, M., Cui, Z.: Offline-to-online: Case-based knowledge distillation with large language models for reinforcement learning. In: ICCBR 2025. LNCS, vol. 15662, pp. 142–156. Springer (2025)
19. Lou, C., Sun, Z., Liang, X., Qu, M., Shen, W., Wang, W., Li, Y., Yang, Q., Wu, S.: Adacot: Pareto-optimal adaptive chain-of-thought triggering via reinforcement learning. arXiv preprint arXiv:2505.11896 (2025)
20. Ma, X., Wan, G., Yu, R., Fang, G., Wang, X.: Cot-valve: Length-compressible chain-of-thought tuning. In: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). pp. 6025–6035 (2025)
21. Mohammadi, M., Li, Y., Lo, J., Yip, W.: Evaluation and benchmarking of llm agents: A survey. In: Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2. pp. 6129–6139 (2025)
22. Patil, S.G., Mao, H., Yan, F., Ji, C.C.J., Suresh, V., Stoica, I., Gonzalez, J.E.: The berkeley function calling leaderboard (BFCL) (2025)
23. Patil, S.G., Zhang, T., Wang, X., Gonzalez, J.E.: Gorilla: Large language model connected with massive apis. In: Advances in Neural Information Processing Systems 37 (NeurIPS 2024) (2024)
24. Qin, Y., Liang, S., Ye, Y., Zhu, K., Yan, L., Lu, Y., Lin, Y., Cong, X., et al.: Toolllm: Facilitating large language models to master 16000+ real-world apis. In: The Twelfth International Conference on Learning Representations (ICLR) (2024)
25. Ruan, J., Chen, Y., Zhang, B., Xu, Z., Bao, T., Mao, H., Li, Z., Zeng, X., Zhao, R., et al.: Tptu: Task planning and tool usage of large language model-based ai agents. In: NeurIPS 2023 foundation models for decision making workshop (2023)
26. Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., Scialom, T.: Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems* **36**, 68539–68551 (2023)
27. Xu, H., Wang, Z., Zhu, Z., Pan, L., Chen, X., Fan, S., Chen, L., Yu, K.: Alignment for efficient tool calling of large language models. In: Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing. pp. 17787–17803 (2025)
28. Yang, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Li, C., Liu, D., Huang, F., et al.: Qwen2.5 technical report (2025), <https://arxiv.org/abs/2412.15115>
29. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K.R., Cao, Y.: React: Synergizing reasoning and acting in language models. In: The eleventh international conference on learning representations (2022)